

Two-Tier File Upload & Display with EC2, S3, and IAM Roles

Course: Cloud Computing **Estimated Time:** 120–150 minutes **Difficulty:** Intermediate **Prerequisites:** Previous lab completed (AWS CLI basics, `.env` pattern, key pair, create/delete script workflow).

1. Learning Objectives

By the end of this lab, you will be able to:

1. Design a **two-tier, decoupled architecture** where two EC2 instances communicate indirectly through Amazon S3 instead of talking to each other directly.
 2. Launch two purpose-built EC2 instances — an **Uploader** and a **Viewer** — each running a small Node.js/Express app whose code is written **locally** and embedded directly into the instance's launch configuration, with nothing fetched over the network at boot.
 3. Create **least-privilege IAM roles** and **instance profiles**, each scoped to the exact S3 action and object its app actually touches (`PutObject` vs. `GetObject` / `ListBucket` on one object key), with **no AWS credentials ever stored on the instance**.
 4. Build and use **EC2 Launch Templates** — reusable, versioned launch configurations — instead of hardcoding every flag into a `run-instances` call, and understand the difference between *creating/versioning* a template and *launching* an instance from it.
 5. Explain why embedding local code directly into user-data — rather than fetching it from Git or S3 at boot — means the instance never needs any code-delivery credential at all.
 6. Extend your `.env` / create / delete script pattern to provision and tear down a **multi-resource stack** (2 EC2 instances + 2 launch templates + 1 S3 bucket + 2 IAM roles + 2 instance profiles) safely and repeatably.
 7. Explain why this design stays within AWS Free Tier limits, and what to monitor if it wouldn't.
-

2. Why This Matters

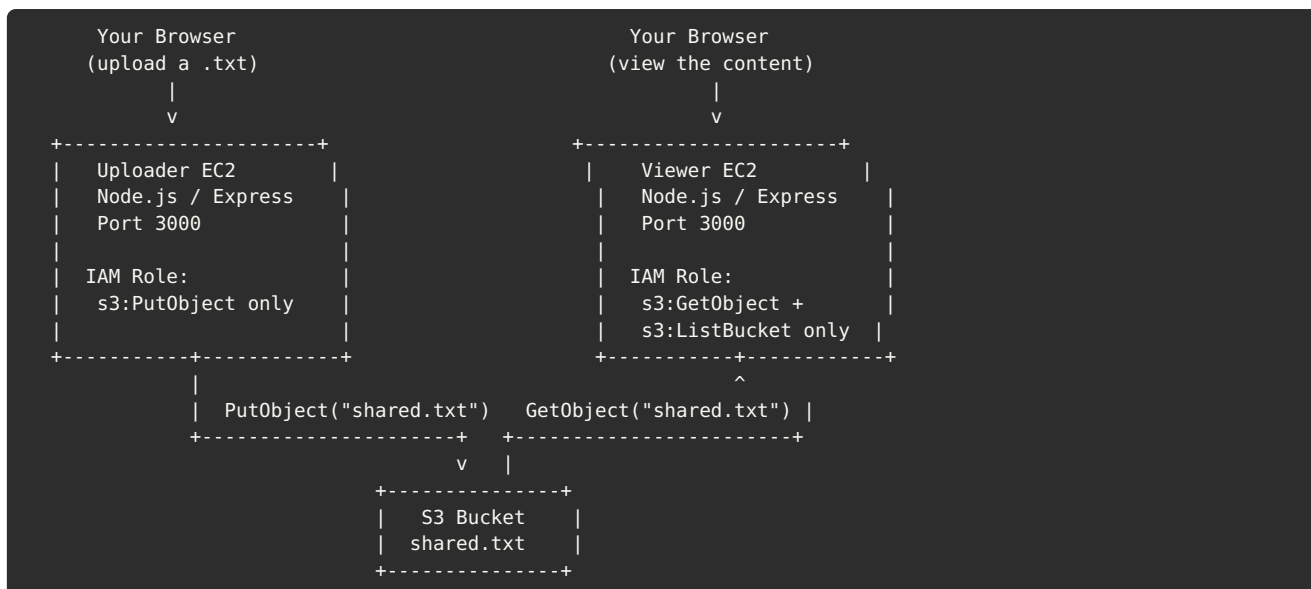
Almost no real cloud application is a single server. Production systems are built from small, single-purpose services that communicate through **shared storage or messaging**, not direct server-to-server logins. Today you'll build the simplest possible version of that pattern:

- An **Uploader** service accepts a `.txt` file from a browser and writes it to S3.
- A **Viewer** service reads that same object from S3 and displays it.
- Neither service ever talks to the other directly, and neither has an AWS access key sitting on disk — each gets **temporary, automatically-rotated credentials** through an **IAM instance role**, scoped to exactly one action.

This mirrors how real teams avoid two of the most common cloud security mistakes: over-permissioned services, and long-lived credentials embedded in code or config files. You'll see the same principle again in Part D, applied to *deploying* the code itself.

⚠ **Cost Warning:** This lab runs **two** EC2 instances simultaneously instead of one. That's twice the hourly burn against your Free Tier's 750 combined `t2.micro` / `t3.micro` hours per month. S3 storage/requests for a single small text file are negligible under the Free Tier (5 GB storage, 20,000 GET, 2,000 PUT per month). **Always run the delete script at the end of this lab** — it's the only step that stops the meter on both instances at once.

3. Architecture Overview



Neither instance has inbound/outbound network rules that let it reach the other directly — the **only** shared resource is the S3 object.

💡 The app **code** doesn't travel through S3 at all — it's written locally in Part C and baked directly into each instance's launch configuration in Part D. The only thing that flows through the bucket at runtime is the one file `shared.txt`.

4. AWS Free-Tier Storage Options — and Why S3

Before creating anything, it's worth knowing what else was on the table. AWS has three storage services that could plausibly hold “the file everyone needs to see” in an architecture like this one — S3, EBS, and EFS — and they're not interchangeable.

⚠️ **Check which Free Tier your account has.** AWS changed its Free Tier program for accounts created on or after **July 15, 2025**. Accounts from before that date get the classic **12-month, per-service** Free Tier (fixed allowances like the ones below). Newer accounts instead get **up to \$200 in credits**, usable on a Free plan for up to 6 months (or until the credits run out), plus a separate set of **Always Free** limits (e.g., DynamoDB, Lambda) that apply regardless of account age. The per-service numbers below describe the classic allowances — if your account is on the credit-based plan, the same usage just draws down your credit balance instead of being free outright, so the cost discipline (delete what you're not using) matters just as much either way.

Service	What it is	Typical Free Tier allowance	Attachment model
S3	Object storage — files reached over an HTTPS API	5 GB storage, 20,000 GET, 2,000 PUT per month	None — any authorized caller reaches it directly, from anywhere
EBS	Block storage — a virtual disk	30 GB storage, 2 million I/Os, 1 GB snapshot storage	Attached to exactly one EC2 instance at a time (standard volumes)
EFS	Managed NFS file system	5 GB storage per month (EFS Standard, One Zone)	Mountable by many instances concurrently, but needs mount targets, a security group for NFS (port 2049), and an NFS client on each instance

Demo: Why EBS Can't Do What We Need

Run these — they work whether or not you currently have any resources running:

```
# See any EBS volumes already in your account and how many instances each is attached to
aws ec2 describe-volumes \
  --query "Volumes[*].[VolumeId,State,Attachments[0].InstanceId]" \
  --output table \
  --region us-east-2

# See your S3 buckets — notice there's no "attachment" column at all
aws s3 ls
```

If you have any EBS volumes from an earlier lab, notice the `describe-volumes` output has **at most one** `InstanceId` per volume — that's not a configuration choice, it's structural. A standard EBS volume physically cannot be attached to two instances simultaneously. `aws s3 ls` has no equivalent column, because S3 objects aren't attached to compute at all — they're just reachable over HTTPS by whoever has the right IAM permissions.

Why S3 for This Lab

- **Both instances need the same object without talking to each other.** S3's HTTPS API is reachable from any instance (or your laptop) at once. EBS fundamentally can't do this without Multi-Attach volumes (io1/io2 only, not Free Tier eligible). EFS *can* do concurrent access, but only after you provision mount targets per subnet and open NFS in your security groups — real plumbing for one small text file.
- **Fine-grained IAM, natively.** S3 lets you write a policy scoped to one exact object key (`shared.txt`), which is exactly the least-privilege pattern from Part B. EBS and EFS control access mostly through the network layer (security groups) and POSIX file permissions, not per-object IAM actions.
- **No mounting, no formatting, no filesystem to manage.** `PutObject` / `GetObject` over HTTPS is all either app needs for its data.
- **Free Tier headroom that matches the workload.** One small text file is nowhere near S3's 5 GB/20k-GET/2k-PUT allowance (or a trivial fraction of a \$200 credit). EBS's per-instance attachment model doesn't even fit the two-writer, two-reader shape of this lab, so its Free Tier allowance isn't really comparable here.

5. Part A — Create the S3 Bucket (Manually, to Understand It)

S3 bucket names must be **globally unique across all of AWS**, not just your account. Pick a name that includes something unique to you.

```
aws s3api create-bucket \
  --bucket itmo-444-544-<your-netid>-fileshare \
  --region us-east-2 \
  --create-bucket-configuration LocationConstraint=us-east-2
```

Confirm it exists:

```
aws s3 ls | grep fileshare
```

✓ **Checkpoint:** Your bucket name appears in the `aws s3 ls` output. Keep it — you'll put it in `.env` in Part E and your create script will re-use it idempotently (it checks before creating).

💡 We're creating it manually once here so you understand the resource before scripting it. Delete it manually too, just this once:

```
aws s3api delete-bucket --bucket itmo-444-544-<your-netid>-fileshare --region us-east-2
```

From Part F onward, bucket creation/deletion is fully automated by your scripts.

6. Part B — Least-Privilege IAM Roles (Manual Walkthrough)

Each instance gets its **own** role with only the one permission it needs. We'll do one by hand so you understand every piece before it's scripted.

B1. Trust Policy (Who Can Assume This Role)

```
cat > trust-policy.json << 'EOF'
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": { "Service": "ec2.amazonaws.com" },
      "Action": "sts:AssumeRole"
    }
  ]
}
EOF
```

This says: “Only the EC2 service itself may assume this role” — not you, not another AWS account.

B2. Create the Uploader Role and Attach a Scoped Policy

```
aws iam create-role \
  --role-name itmo-444-544-uploader-role \
  --assume-role-policy-document file://trust-policy.json

cat > uploader-policy.json << EOF
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": ["s3:PutObject"],
      "Resource": "arn:aws:s3:::itmo-444-544-<your-netid>-fileshare/shared.txt"
    }
  ]
}
EOF

aws iam put-role-policy \
  --role-name itmo-444-544-uploader-role \
  --policy-name uploader-put-only \
  --policy-document file://uploader-policy.json
```

Notice the policy names one **exact object key** — not `bucket/*`. This role can write exactly one object and nothing else; it can't list the bucket, read anything back, or touch any other object. If this instance were compromised, an attacker could overwrite `shared.txt` but could not read any other object in your account.

B3. Wrap the Role in an Instance Profile

EC2 instances attach to **instance profiles**, not roles directly (a profile is a thin wrapper around one role).

```
aws iam create-instance-profile \
  --instance-profile-name itmo-444-544-uploader-profile

aws iam add-role-to-instance-profile \
  --instance-profile-name itmo-444-544-uploader-profile \
  --role-name itmo-444-544-uploader-role
```

✓ **Checkpoint:** `aws iam get-instance-profile --instance-profile-name itmo-444-544-uploader-profile` shows your role attached under `Roles`.

💡 In Part F, your create script does this same three-step dance (trust policy → role + inline policy → instance profile) for **both** the uploader and viewer roles, and checks whether each already exists before creating it — so re-running the script is safe.

7. Part C — The Two Node.js Apps (Written Locally)

These apps live entirely on your local machine, in a `lab03/` folder. Nothing here gets uploaded anywhere by hand — Part D shows how the create script picks these files up directly and bakes them into each instance's launch configuration.

```
lab03/
├── uploader-app/
│   ├── app.js
│   └── package.json
└── viewer-app/
    ├── app.js
    └── package.json
```

C1. Uploader App Logic

The uploader serves an HTML form and, on submit, streams the file straight to S3 — it never writes the upload to the instance's own disk. It also **validates server-side**, rejecting anything that isn't a `.txt` file and anything over **1MB**, both enforced by `multer` before the file ever reaches S3.

```
mkdir -p lab03/uploader-app
cd lab03/uploader-app
```

`package.json` :

```
{
  "name": "uploader-app",
  "version": "1.0.0",
  "main": "app.js",
  "dependencies": {
    "express": "^4.19.2",
    "multer": "^1.4.5-lts.1",
    "@aws-sdk/client-s3": "^3.600.0"
  }
}
```

`app.js` :

```

const express = require('express');
const multer = require('multer');
const { S3Client, PutObjectCommand } = require('@aws-sdk/client-s3');

const app = express();
const MAX_FILE_SIZE = 1 * 1024 * 1024; // 1MB (1,048,576 bytes)

const upload = multer({
  storage: multer.memoryStorage(),
  limits: { fileSize: MAX_FILE_SIZE },
  fileFilter: (req, file, cb) => {
    const isTxt = file.mimetype === 'text/plain' && file.originalname.toLowerCase().endsWith('.txt');
    if (!isTxt) {
      return cb(new Error('ONLY_TXT_ALLOWED'));
    }
    cb(null, true);
  },
});

const s3 = new S3Client({ region: process.env.AWS_REGION });
const BUCKET = process.env.BUCKET_NAME;
const PORT = process.env.PORT || 3000;

app.get('/', (req, res) => {
  res.send(`
    <h1>Uploader</h1>
    <form method="POST" action="/upload" enctype="multipart/form-data">
      <input type="file" name="txtfile" accept=".txt" required>
      <p>Only .txt files, max 1MB</p>
      <button type="submit">Upload</button>
    </form>
  `);
});

app.post('/upload', (req, res) => {
  upload.single('txtfile')(req, res, async (err) => {
    if (err) {
      if (err.code === 'LIMIT_FILE_SIZE') {
        return res.status(413).send(`<p>Upload failed: file exceeds the 1MB limit.</p><a href="/">Try
again</a>`);
      }
      if (err.message === 'ONLY_TXT_ALLOWED') {
        return res.status(415).send(`<p>Upload failed: only .txt files are allowed.</p><a href="/">Try
again</a>`);
      }
      return res.status(400).send(`<p>Upload failed: ' + err.message + '</p><a href="/">Try again</a>`);
    }
    if (!req.file) return res.status(400).send('No file selected.');
```

```

    try {
      await s3.send(new PutObjectCommand({
        Bucket: BUCKET,
        Key: 'shared.txt',
        Body: req.file.buffer,
        ContentType: 'text/plain',
      }));
      res.send(`<p>Upload successful.</p><a href="/">Upload another</a>`);
    } catch (err2) {
      res.status(500).send('Upload failed: ' + err2.message);
    }
  });
});

app.listen(PORT, () => console.log(`Uploader listening on ${PORT}`));

```

💡 `upload.single('txtfile')` is normally used as declarative middleware (`app.post('/upload', upload.single('txtfile'), handler)`), but that form has no clean way to catch a `fileFilter` or `LIMIT_FILE_SIZE` error inline. Calling it as a function — `upload.single('txtfile')(req, res, callback)` — gives you that error in the callback directly, so you can return a specific message instead of Express's generic default error page.

△ The HTML form's `accept=".txt"` only changes what the browser's file picker *suggests* — it's a UX hint, not a security control. Anyone can rename a file to end in `.txt`, or bypass the picker with a raw HTTP request, and send whatever bytes they want. That's why `fileFilter` re-checks `file.mimetype` **and** the file extension on the server before accepting anything — the same “don't trust the client” instinct that motivates validating input anywhere else in a real application. Checking both the declared MIME type and the extension catches more spoofing attempts than either alone, though a determined attacker could still fake both; a production system handling untrusted uploads would also inspect the file's actual byte content, which is out of scope here.

Notice there is **no** `accessKeyId` **or** `secretAccessKey` **anywhere** — `S3Client` automatically picks up temporary credentials from the instance's attached IAM role.

C2. Viewer App Logic

The viewer reads the same object back and renders it as plain text.

```
cd ../../
mkdir -p lab03/viewer-app
cd lab03/viewer-app
```

`package.json` :

```
{
  "name": "viewer-app",
  "version": "1.0.0",
  "main": "app.js",
  "dependencies": {
    "express": "^4.19.2",
    "@aws-sdk/client-s3": "^3.600.0"
  }
}
```

`app.js` :

```
const express = require('express');
const { S3Client, GetObjectCommand } = require('@aws-sdk/client-s3');

const app = express();
const s3 = new S3Client({ region: process.env.AWS_REGION });
const BUCKET = process.env.BUCKET_NAME;
const PORT = process.env.PORT || 3000;

const streamToString = (stream) => new Promise((resolve, reject) => {
  const chunks = [];
  stream.on('data', (c) => chunks.push(c));
  stream.on('error', reject);
  stream.on('end', () => resolve(Buffer.concat(chunks).toString('utf-8')));
});

app.get('/', async (req, res) => {
  try {
    const data = await s3.send(new GetObjectCommand({ Bucket: BUCKET, Key: 'shared.txt' }));
    const content = await streamToString(data.Body);
    res.send(`<h1>Viewer</h1><pre>${content}</pre><a href="/">Refresh</a>`);
  } catch (err) {
    res.send(`<h1>Viewer</h1><p>No file uploaded yet.</p><a href="/">Refresh</a>`);
  }
});

app.listen(PORT, () => console.log(`Viewer listening on ${PORT}`));
```

✓ **Checkpoint:** You can explain, in one sentence, why the viewer's IAM role does not need `s3:PutObject`.

💡 It's fine (even good practice) to also commit `lab03/` to a Git repo for version history — but that's just for your own record-keeping. Nothing in Part D reads from Git; the create script reads straight from the files sitting in this

local folder.

8. Part D — Launch Templates: Embedding Local Code into User-Data

So far in this course, `run-instances` calls have listed every setting inline: AMI, instance type, key pair, security group, and so on. A **Launch Template** is a saved, versioned bundle of those same settings — you create (or update) the template once, then launch instances *from* it. Real teams use this so “what does an uploader instance look like” has one authoritative definition, and so tools like Auto Scaling Groups can reference it without repeating every flag.

Crucially for this lab, a launch template’s `UserData` field can hold the **entire boot script**, including your app code — so there’s no separate fetch step (no `git clone`, no `aws s3 cp` for code) at boot at all. The instance already has everything it needs the moment it starts.

D1. How Local Files Become User-Data

The create script (Part F) will, for each app:

1. Read `lab03/uploader-app/app.js` and `package.json` straight off your local disk.
2. **Base64-encode** each file. This isn’t obfuscation — it’s so that quotes, `${...}` JavaScript template literals, and newlines inside the code can never be misread as shell syntax while the script is being assembled.
3. Write a user-data script that decodes them back into real files on the instance, installs Node, runs `npm install`, and starts the app as a `systemd` service.

A minimal illustration of the mechanism (the full version is in Part F):

```
APP_JS_B64=$(base64 -w0 lab03/uploader-app/app.js)
echo "echo \"\$APP_JS_B64\" | base64 -d > /opt/app/app.js" >> user-data-uploader.sh
```

At boot, the instance just runs that one decoded line, recreating `app.js` exactly as it exists on your machine right now — no network fetch of code, no credential of any kind involved.

D2. Launch Template Data (the JSON Shape)

`create-launch-template` takes a JSON document describing the instance to launch:

```
{
  "ImageId": "ami-xxxxxxxxxxxxxxxx",
  "InstanceType": "t3.micro",
  "KeyName": "itmo-444-544-lab-keypair",
  "SecurityGroupIds": ["sg-xxxxxxxxxxxxxxxx"],
  "IamInstanceProfile": { "Name": "itmo-444-544-uploader-profile" },
  "UserData": "<base64-encoded-entire-user-data-script>",
  "TagSpecifications": [
    { "ResourceType": "instance", "Tags": [{ "Key": "Name", "Value": "uploader-app" }] }
  ]
}
```

△ Unlike `run-instances --user-data file://path` (which base64-encodes the file for you automatically), `create-launch-template`’s `UserData` field must **already be base64-encoded** by you before it goes into the JSON — it’s just a plain string field as far as the API is concerned.

D3. Creating a Template vs. Launching From One

These are two different API calls, and it’s worth keeping them straight:

```
# 1. Create the template — this does NOT launch anything
aws ec2 create-launch-template \
  --launch-template-name itmo-444-544-uploader-lt \
  --launch-template-data file://uploader-lt-data.json \
  --region us-east-2

# 2. Launch an instance FROM the template — this is what actually starts a machine
aws ec2 run-instances \
  --launch-template LaunchTemplateName=itmo-444-544-uploader-lt,Version='$Latest' \
  --count 1 \
  --region us-east-2
```

If you re-run `create-launch-template` with the same name, AWS rejects it — templates are versioned, not overwritten. To publish an update (say, you edited `app.js`), you call `create-launch-template-version` instead, which adds a new numbered version without touching the old one. `Version='$Latest'` in `run-instances` always picks up whichever version was most recently added — which is exactly what your create script needs, since you'll want every re-run to reflect your current local files. Part F implements this check-then-version-or-create logic in full.

9. Part E — Security Group and Updated `.env`

E1. Create a Security Group for the App Tier

```
aws ec2 create-security-group \
  --group-name itmo-444-544-app-sg \
  --description "Allow SSH and app port 3000" \
  --region us-east-2

aws ec2 authorize-security-group-ingress \
  --group-id <sg-id> --protocol tcp --port 22 --cidr 0.0.0.0/0 --region us-east-2

aws ec2 authorize-security-group-ingress \
  --group-id <sg-id> --protocol tcp --port 3000 --cidr 0.0.0.0/0 --region us-east-2
```

E2. Extend `.env`

Add these keys to your existing `.env` (keep the ones from the previous lab — `AWS_REGION`, `AMI_ID`, `KEY_NAME`, `INSTANCE_TYPE`):

```
BUCKET_NAME=itmo-444-544-<your-netid>-fileshare
APP_SECURITY_GROUP_ID=sg-xxxxxxxxxxxxxxxx
UPLOADER_ROLE_NAME=itmo-444-544-uploader-role
UPLOADER_PROFILE_NAME=itmo-444-544-uploader-profile
UPLOADER_LT_NAME=itmo-444-544-uploader-lt
VIEWER_ROLE_NAME=itmo-444-544-viewer-role
VIEWER_PROFILE_NAME=itmo-444-544-viewer-profile
VIEWER_LT_NAME=itmo-444-544-viewer-lt
APP_PORT=3000
```

Also add the equivalent placeholder lines to `.env.example`, and confirm `.env` is still excluded via `.gitignore` (`git status` should never show it).

10. Part F — Create and Delete Scripts for the Full Stack

These scripts are **idempotent** — safe to re-run without duplicating resources — and manage all the resource types together: 1 S3 bucket, 2 IAM roles, 2 instance profiles, 2 launch templates, 2 EC2 instances. Re-running `create_app_stack.sh` after editing your local `lab03/` code publishes a new launch template version and launches fresh instances from it.

F1. `create_app_stack.sh`

Run this from the folder that contains your `lab03/` directory.

```

#!/bin/bash
set -e
export $(grep -v '^#' .env | xargs)

echo "== S3 bucket =="
if aws s3api head-bucket --bucket "$BUCKET_NAME" --region "$AWS_REGION" 2>/dev/null; then
    echo "Bucket already exists, skipping."
else
    aws s3api create-bucket --bucket "$BUCKET_NAME" --region "$AWS_REGION" \
        --create-bucket-configuration LocationConstraint="$AWS_REGION"
fi

cat > trust-policy.json << 'EOF'
{ "Version": "2012-10-17", "Statement": [
    { "Effect": "Allow", "Principal": { "Service": "ec2.amazonaws.com" }, "Action": "sts:AssumeRole" } ] }
EOF

echo "== Uploader role (PutObject on shared.txt only) =="
aws iam create-role --role-name "$UPLOADER_ROLE_NAME" \
    --assume-role-policy-document file://trust-policy.json 2>/dev/null || echo "Role exists, skipping."
cat > uploader-policy.json << EOF
{ "Version": "2012-10-17", "Statement": [
    { "Effect": "Allow", "Action": ["s3:PutObject"], "Resource": "arn:aws:s3:::${BUCKET_NAME}/shared.txt" } ] }
EOF
aws iam put-role-policy --role-name "$UPLOADER_ROLE_NAME" \
    --policy-name "${UPLOADER_ROLE_NAME}-policy" --policy-document file://uploader-policy.json
aws iam create-instance-profile --instance-profile-name "$UPLOADER_PROFILE_NAME" 2>/dev/null || echo "Profile exists, skipping."
aws iam add-role-to-instance-profile --instance-profile-name "$UPLOADER_PROFILE_NAME" \
    --role-name "$UPLOADER_ROLE_NAME" 2>/dev/null || true

echo "== Viewer role (GetObject + ListBucket on shared.txt only) =="
aws iam create-role --role-name "$VIEWER_ROLE_NAME" \
    --assume-role-policy-document file://trust-policy.json 2>/dev/null || echo "Role exists, skipping."
cat > viewer-policy.json << EOF
{ "Version": "2012-10-17", "Statement": [
    { "Effect": "Allow", "Action": ["s3:GetObject"], "Resource": "arn:aws:s3:::${BUCKET_NAME}/shared.txt" },
    { "Effect": "Allow", "Action": ["s3:ListBucket"], "Resource": "arn:aws:s3:::${BUCKET_NAME}" } ] }
EOF
aws iam put-role-policy --role-name "$VIEWER_ROLE_NAME" \
    --policy-name "${VIEWER_ROLE_NAME}-policy" --policy-document file://viewer-policy.json
aws iam create-instance-profile --instance-profile-name "$VIEWER_PROFILE_NAME" 2>/dev/null || echo "Profile exists, skipping."
aws iam add-role-to-instance-profile --instance-profile-name "$VIEWER_PROFILE_NAME" \
    --role-name "$VIEWER_ROLE_NAME" 2>/dev/null || true

echo "Waiting 15s for IAM propagation..."
sleep 15

echo "== Building user-data from local lab03/ files =="
UPLOADER_APP_JS_B64=$(base64 -w0 lab03/uploader-app/app.js)
UPLOADER_PKG_JSON_B64=$(base64 -w0 lab03/uploader-app/package.json)

cat > user-data-uploader.sh << EOF
#!/bin/bash
dnf update -y
dnf install -y nodejs
mkdir -p /opt/app
echo "$UPLOADER_APP_JS_B64" | base64 -d > /opt/app/app.js
echo "$UPLOADER_PKG_JSON_B64" | base64 -d > /opt/app/package.json
cd /opt/app
npm install --omit=dev

cat > /etc/systemd/system/uploader.service << 'SERVICE'
[Unit]
Description=Uploader App
After=network.target

[Service]
Environment=BUCKET_NAME=$BUCKET_NAME
Environment=AWS_REGION=$AWS_REGION
Environment=PORT=3000
WorkingDirectory=/opt/app

```

```

ExecStart=/usr/bin/node /opt/app/app.js
Restart=always
User=ec2-user

[Install]
WantedBy=multi-user.target
SERVICE

chown -R ec2-user:ec2-user /opt/app
systemctl daemon-reload
systemctl enable uploader.service
systemctl start uploader.service
EOF

VIEWER_APP_JS_B64=$(base64 -w0 lab03/viewer-app/app.js)
VIEWER_PKG_JSON_B64=$(base64 -w0 lab03/viewer-app/package.json)

cat > user-data-viewer.sh << EOF
#!/bin/bash
dnf update -y
dnf install -y nodejs
mkdir -p /opt/app
echo "$VIEWER_APP_JS_B64" | base64 -d > /opt/app/app.js
echo "$VIEWER_PKG_JSON_B64" | base64 -d > /opt/app/package.json
cd /opt/app
npm install --omit=dev

cat > /etc/systemd/system/viewer.service << 'SERVICE'
[Unit]
Description=Viewer App
After=network.target

[Service]
Environment=BUCKET_NAME=$BUCKET_NAME
Environment=AWS_REGION=$AWS_REGION
Environment=PORT=3000
WorkingDirectory=/opt/app
ExecStart=/usr/bin/node /opt/app/app.js
Restart=always
User=ec2-user

[Install]
WantedBy=multi-user.target
SERVICE

chown -R ec2-user:ec2-user /opt/app
systemctl daemon-reload
systemctl enable viewer.service
systemctl start viewer.service
EOF

echo "== Building launch template data =="
UPLOADER_USERDATA_B64=$(base64 -w0 user-data-uploader.sh)
VIEWER_USERDATA_B64=$(base64 -w0 user-data-viewer.sh)

cat > uploader-lt-data.json << EOF
{
  "ImageId": "$AMI_ID",
  "InstanceType": "$INSTANCE_TYPE",
  "KeyName": "$KEY_NAME",
  "SecurityGroupIds": ["$APP_SECURITY_GROUP_ID"],
  "IamInstanceProfile": { "Name": "$UPLOADER_PROFILE_NAME" },
  "UserData": "$UPLOADER_USERDATA_B64",
  "TagSpecifications": [
    { "ResourceType": "instance", "Tags": [{ "Key": "Name", "Value": "uploader-app" }] }
  ]
}
EOF

cat > viewer-lt-data.json << EOF
{
  "ImageId": "$AMI_ID",
  "InstanceType": "$INSTANCE_TYPE",

```

```

        "KeyName": "$KEY_NAME",
        "SecurityGroupIds": ["$APP_SECURITY_GROUP_ID"],
        "IamInstanceProfile": { "Name": "$VIEWER_PROFILE_NAME" },
        "UserData": "$VIEWER_USERDATA_B64",
        "TagSpecifications": [
            { "ResourceType": "instance", "Tags": [{ "Key": "Name", "Value": "viewer-app" }] }
        ]
    }
}
EOF

echo "== Creating/updating launch templates =="
if aws ec2 describe-launch-templates --launch-template-names "$UPLOADER_LT_NAME" --region "$AWS_REGION"
>/dev/null 2>&1; then
    echo "Template $UPLOADER_LT_NAME exists – publishing a new version"
    aws ec2 create-launch-template-version \
        --launch-template-name "$UPLOADER_LT_NAME" \
        --source-version 1 \
        --launch-template-data file://uploader-lt-data.json \
        --region "$AWS_REGION" > /dev/null
else
    aws ec2 create-launch-template \
        --launch-template-name "$UPLOADER_LT_NAME" \
        --launch-template-data file://uploader-lt-data.json \
        --region "$AWS_REGION" > /dev/null
fi

if aws ec2 describe-launch-templates --launch-template-names "$VIEWER_LT_NAME" --region "$AWS_REGION"
>/dev/null 2>&1; then
    echo "Template $VIEWER_LT_NAME exists – publishing a new version"
    aws ec2 create-launch-template-version \
        --launch-template-name "$VIEWER_LT_NAME" \
        --source-version 1 \
        --launch-template-data file://viewer-lt-data.json \
        --region "$AWS_REGION" > /dev/null
else
    aws ec2 create-launch-template \
        --launch-template-name "$VIEWER_LT_NAME" \
        --launch-template-data file://viewer-lt-data.json \
        --region "$AWS_REGION" > /dev/null
fi

echo "== Launching instances from the templates =="
UPLOADER_ID=$(aws ec2 run-instances \
    --launch-template "LaunchTemplateName=$UPLOADER_LT_NAME,Version=\$Latest" \
    --count 1 \
    --region "$AWS_REGION" \
    --query "Instances[0].InstanceId" --output text)

VIEWER_ID=$(aws ec2 run-instances \
    --launch-template "LaunchTemplateName=$VIEWER_LT_NAME,Version=\$Latest" \
    --count 1 \
    --region "$AWS_REGION" \
    --query "Instances[0].InstanceId" --output text)

echo "$UPLOADER_ID" > uploader_instance_id.txt
echo "$VIEWER_ID" > viewer_instance_id.txt

echo "Waiting for both instances to reach 'running'..."
aws ec2 wait instance-running --instance-ids "$UPLOADER_ID" "$VIEWER_ID" --region "$AWS_REGION"

aws ec2 describe-instances --instance-ids "$UPLOADER_ID" "$VIEWER_ID" \
    --query "Reservations[*].Instances[*].[Tags[?Key=='Name'].Value|[0],PublicIpAddress]" \
    --output table --region "$AWS_REGION"

echo "Instances are running. Give them ~60-90s to finish installing Node.js before browsing to :3000."

```

```
chmod +x create_app_stack.sh
```

F2. delete_app_stack.sh

```
#!/bin/bash
set -e
export $(grep -v '^#' .env | xargs)

if [[ -f uploader_instance_id.txt && -f viewer_instance_id.txt ]]; then
    UPLOADER_ID=$(cat uploader_instance_id.txt)
    VIEWER_ID=$(cat viewer_instance_id.txt)
    echo "Terminating instances: $UPLOADER_ID $VIEWER_ID"
    aws ec2 terminate-instances --instance-ids "$UPLOADER_ID" "$VIEWER_ID" --region "$AWS_REGION"
    aws ec2 wait instance-terminated --instance-ids "$UPLOADER_ID" "$VIEWER_ID" --region "$AWS_REGION"
    echo "Instances terminated."
    rm -f uploader_instance_id.txt viewer_instance_id.txt
else
    echo "No instance ID files found – skipping instance termination."
fi

echo "== Deleting launch templates =="
aws ec2 delete-launch-template --launch-template-name "$UPLOADER_LT_NAME" --region "$AWS_REGION" 2>/dev/null || true
aws ec2 delete-launch-template --launch-template-name "$VIEWER_LT_NAME" --region "$AWS_REGION" 2>/dev/null || true

echo "== Removing IAM instance profiles and roles =="
aws iam remove-role-from-instance-profile --instance-profile-name "$UPLOADER_PROFILE_NAME" \
    --role-name "$UPLOADER_ROLE_NAME" 2>/dev/null || true
aws iam delete-instance-profile --instance-profile-name "$UPLOADER_PROFILE_NAME" 2>/dev/null || true
aws iam delete-role-policy --role-name "$UPLOADER_ROLE_NAME" \
    --policy-name "${UPLOADER_ROLE_NAME}-policy" 2>/dev/null || true
aws iam delete-role --role-name "$UPLOADER_ROLE_NAME" 2>/dev/null || true

aws iam remove-role-from-instance-profile --instance-profile-name "$VIEWER_PROFILE_NAME" \
    --role-name "$VIEWER_ROLE_NAME" 2>/dev/null || true
aws iam delete-instance-profile --instance-profile-name "$VIEWER_PROFILE_NAME" 2>/dev/null || true
aws iam delete-role-policy --role-name "$VIEWER_ROLE_NAME" \
    --policy-name "${VIEWER_ROLE_NAME}-policy" 2>/dev/null || true
aws iam delete-role --role-name "$VIEWER_ROLE_NAME" 2>/dev/null || true

echo "== Emptying and deleting S3 bucket =="
aws s3 rm "s3://$BUCKET_NAME" --recursive --region "$AWS_REGION" 2>/dev/null || true
aws s3api delete-bucket --bucket "$BUCKET_NAME" --region "$AWS_REGION" 2>/dev/null || true

echo "Cleanup complete."
```

```
chmod +x delete_app_stack.sh
```

✓ **Checkpoint:** `create_app_stack.sh` prints a table with two `running` instances and their public IPs; `delete_app_stack.sh` reports every resource removed with no errors on a second run.

11. Part G — End-to-End Test

1. Run `./create_app_stack.sh` and copy both public IPs from the output table.
2. In a browser, open `http://<uploader-public-ip>:3000`. Upload any `.txt` file.
3. In another tab, open `http://<viewer-public-ip>:3000`. You should see the file's contents rendered on the page.
4. Upload a **different** `.txt` file on the uploader, then click **Refresh** on the viewer page — the content should update, proving the viewer is reading live from S3, not caching locally.
5. Create (or find) a `.txt` file larger than 1MB and try uploading it on the uploader — confirm you get the “file exceeds the 1MB limit” error instead of a successful upload, and that the viewer's content is unchanged after refreshing.
6. Try uploading a non-`.txt` file (e.g., a `.jpg` or `.pdf`, renamed or not) — confirm you get the “only `.txt` files are allowed” error, and that the viewer's content is still unchanged.
7. Edit `lab03/uploader-app/app.js` (e.g., change the `<h1>` text), re-run `./create_app_stack.sh`, and confirm the uploader page reflects your edit — this proves the launch template published a new version and the new instance launched from it.
8. Run `./delete_app_stack.sh` and confirm both public IPs stop responding.

✓ **Checkpoint:** Steps 2–7 work without ever SSHing into either instance, without Git, and without any code-fetch step at boot.

12. Troubleshooting

Issue	Likely Cause	Fix
Browser can't reach <code>:3000</code> at all	Security group missing port 3000, or app still installing	Recheck <code>APP_SECURITY_GROUP_ID</code> rules; wait 60–90s after launch for <code>npm install</code> to finish
Upload fails with “file exceeds the 1MB limit”	The <code>.txt</code> file you selected is larger than <code>MAX_FILE_SIZE</code> in <code>app.js</code>	Expected behavior — pick a smaller file, or raise <code>MAX_FILE_SIZE</code> in the uploader's <code>app.js</code> and re-run <code>create_app_stack.sh</code> to publish a new template version
Upload fails with “only .txt files are allowed”	You selected a file whose extension or MIME type isn't <code>.txt / text/plain</code>	Expected behavior — the <code>fileFilter</code> in <code>app.js</code> is rejecting it server-side, independent of what the browser's file picker allowed through
<code>base64: lab03/uploader-app/app.js: No such file or directory</code>	<code>create_app_stack.sh</code> run from the wrong working directory	<code>cd</code> to the folder that contains <code>lab03/</code> before running the script
Uploader/Viewer page doesn't reflect a code change you just made	Ran the create script from the wrong directory, or forgot to save the file before running it	Confirm the edit is saved, re-run <code>create_app_stack.sh</code> , and check <code>aws ec2 describe-launch-template-versions</code> for a new version number
<code>run-instances --launch-template</code> error: “unable to resolve version”	<code>\$Latest</code> wasn't escaped and your local shell expanded it to an empty string	Use <code>\\$Latest</code> (escaped) inside double-quoted strings, exactly as in the script
<code>InvalidLaunchTemplateName.AlreadyExistsException</code>	Script tried <code>create-launch-template</code> on a name that already exists	Shouldn't happen with the script as written — it checks first and calls <code>create-launch-template-version</code> instead; if you see this, you likely ran the raw <code>aws ec2 create-launch-template</code> command by hand
Viewer shows “No file uploaded yet” forever	Upload never completed, or wrong bucket name in <code>.env</code>	Re-check <code>BUCKET_NAME</code> matches on both <code>.env</code> and what you created; retry the upload
<code>AccessDenied</code> on upload	Uploader role/policy not attached, or IAM propagation not finished	Re-run <code>create_app_stack.sh</code> (idempotent); it includes a 15s propagation wait
<code>run-instances</code> fails: “Invalid IAM Instance Profile”	Instance profile created seconds before use; IAM hasn't propagated yet	Increase the <code>sleep 15</code> in the script, or re-run — role/profile creation is skipped if it already exists
<code>npm install</code> fails on the instance	No internet route (custom VPC/subnet with no IGW)	Confirm you're using the account's default VPC , which has an internet gateway and auto-assigns public IPs
Uploader/Viewer works once, then stops after a reboot	N/A — this shouldn't happen	The <code>systemd</code> service (<code>Restart=always</code> , <code>enabled</code>) should keep it running; check <code>systemctl status uploader.service</code> over SSH if it does

Two Free-Tier instances push you over 750 hrs for the month	Left running too long across multiple lab sessions	Track total EC2 hours in AWS Billing; always run <code>delete_app_stack.sh</code> promptly
<code>delete_app_stack.sh</code> errors on IAM delete	Role/profile already removed, or attached in a different order	Errors here are suppressed with <code> true</code> intentionally — verify manually with <code>aws iam list-roles</code> if unsure

13. Submission Checklist

- ☐ Screenshot of your local `lab03/uploader-app/` and `lab03/viewer-app/` folders showing `app.js` and `package.json` in each.
- ☐ Screenshot of the Uploader page and a successful upload confirmation.
- ☐ Screenshot of attempting to upload a `.txt` file larger than 1MB, showing the “file exceeds the 1MB limit” error.
- ☐ Screenshot of attempting to upload a non-`.txt` file (e.g., a `.jpg` or `.pdf`), showing the “only .txt files are allowed” error.
- ☐ Screenshot of the Viewer page displaying the uploaded file’s content.
- ☐ Screenshot of the Viewer page **after** uploading a second, different file and clicking Refresh, showing the updated content.
- ☐ Screenshot of `aws ec2 describe-launch-templates` showing both templates.
- ☐ Screenshot of `aws ec2 describe-launch-template-versions --launch-template-name <uploader-lt-name>` after re-running the create script twice, showing more than one version.
- ☐ `aws iam get-role-policy` output (or screenshot) for both roles, showing each has only its one intended action on the exact `shared.txt` key.
- ☐ Screenshot of `create_app_stack.sh` output showing both instances `running` with public IPs.
- ☐ Screenshot of `delete_app_stack.sh` completing with no errors.
- ☐ One paragraph explaining why the uploader and viewer use **separate** IAM roles instead of one shared role with both permissions.
- ☐ One or two sentences explaining why embedding code directly into user-data means the instance never needs a Git or S3 credential just to fetch its own code.

14. Key Terms Glossary

- **IAM Role** – An identity with permissions that can be *assumed* by a trusted principal (here, the EC2 service) instead of being tied to a fixed user with long-lived keys.
- **Instance Profile** – A container that lets an IAM role be attached to an EC2 instance at launch.
- **Least Privilege** – Granting only the exact permissions a resource needs to do its job, nothing more.
- **Trust Policy** – The IAM document defining *who* (which principal) is allowed to assume a role.
- **Decoupled Architecture** – A design where components communicate through shared state (like S3) rather than directly, so either side can change or fail independently.
- **Launch Template** – A saved, versioned EC2 launch configuration (AMI, instance type, security groups, IAM profile, user-data, tags) that instances are launched *from*, rather than specifying every setting inline.
- **Launch Template Version** – A numbered snapshot of a launch template’s settings; creating a new version never modifies old ones, and `run-instances` can target `$Latest`, `$Default`, or a specific number.
- **systemd service** – A managed background process on Linux that can auto-start on boot and auto-restart on failure.
- **Idempotent script** – A script safe to run multiple times without creating duplicate resources or erroring out.